

Program 2: 1D Array Searching

Aim

To write a C program that searches for an element in a one-dimensional array using linear search and binary search.

Theory

Searching is the process of finding the location of a given element (the key) in a collection of data.

Linear Search compares the key with each element from the first to the last. It works on unsorted arrays. Worst-case time complexity: $O(n)$.

Binary Search works only on a *sorted* array. It compares the key with the middle element; if they differ, it discards the half that cannot contain the key and repeats. Time complexity: $O(\log n)$.

Both methods return the index of the key, or -1 if the key is not present.

Algorithm

1. Read the array and the key to be searched.
2. Linear search: for $i = 0$ to $n-1$, if $a[i] == \text{key}$, return i . If the loop ends, return -1.
3. Sort the array (bubble sort is used here).
4. Binary search: set $\text{low} = 0$, $\text{high} = n-1$. While $\text{low} \leq \text{high}$: $\text{mid} = (\text{low} + \text{high}) / 2$; if $a[\text{mid}] == \text{key}$ return mid ; if $a[\text{mid}] < \text{key}$ set $\text{low} = \text{mid} + 1$; else set $\text{high} = \text{mid} - 1$.
5. Print the result of both searches.

C Source Code

```
/*
 * Program : 1D Array Searching (Linear Search and Binary Search)
 *
 * THEORY:
 * - Linear Search : compare the key with each element one by one.
 *                   Works on any array.           Time  $O(n)$ 
 * - Binary Search : repeatedly halve the search range by comparing the
 *                   key with the middle element.
 *                   Requires a SORTED array.       Time  $O(\log n)$ 
 * - Both return the index of the key, or -1 if it is not found.
 */
#include <stdio.h>
#define MAX 100

int linearSearch(int a[], int n, int key) {
    for (int i = 0; i < n; i++)
        if (a[i] == key)
            return i;
    return -1;
}

int binarySearch(int a[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == key) return mid;
        else if (a[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

void sortArray(int a[], int n) { /* simple bubble sort */
```

```

        for (int i = 0; i < n - 1; i++)
            for (int j = 0; j < n - i - 1; j++)
                if (a[j] > a[j + 1]) {
                    int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                }
    }

int main() {
    int a[MAX], n, key, res;

    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter element to search: ");
    scanf("%d", &key);

    res = linearSearch(a, n, key);
    if (res != -1) printf("Linear Search : Found at index %d (position %d)\n", res, res + 1);
    else
        printf("Linear Search : Element not found\n");

    sortArray(a, n);                /* binary search needs sorted data */
    printf("Sorted array : ");
    for (int i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");

    res = binarySearch(a, n, key);
    if (res != -1) printf("Binary Search : Found at index %d (position %d) of sorted array\n", res, res + 1);
    else
        printf("Binary Search : Element not found\n");
    return 0;
}

```

Sample Input

Input: 6 elements (45 12 89 33 7 60), key = 33.

Sample Output (program run with the input above)

```

Enter number of elements: Enter 6 elements: Enter element to search: Linear Search : Found at index 3 ({}
Sorted array : 7 12 33 45 60 89
Binary Search : Found at index 2 (position 3) of sorted array

```